

Prácticas de OOP

Combinando OOP, Git, Gitlab y Pruebas unitarias

Posible definición de los requisitos

Los empleados tienen un nombre y pagan impuestos según una tasa del 30%.
Los impuestos se calculan sobre la nómina del empleado. Cada empleado tiene una nómina diferente.

Se pide hacer un programa que calcule la nómina de dos personas (Pepe y Ana) que ganan 20.000 y 30.000 respectivamente.

¿Qué opináis de este código?

```
nominal = 20000  
nombre1 = "Pepe"
```

```
def CalculoImpuestos (nombre, nom):  
    impuestos = nom*0.30  
    print ("El empleado {name} debe pagar {tax:.2f}".format(name=nombre,  
        tax=impuestos))  
    return impuestos  
  
def displayCost(total):  
    print("Los impuestos a pagar en total son {:.2f} euros".format(total))
```

```
nomina2 =30000  
nombre2 = "Ana"  
displayCost(CalculoImpuestos(nombre1, nominal) + CalculoImpuestos(nombre2,  
nomina2))
```

¿Qué opináis de este código?

Dar vuestra opinión sobre los siguientes aspectos:

- ¿Sigue una guía de estilo razonable? (puedes utilizar `pycodestyle`)
- ¿Es legible?
- ¿Se puede reutilizar el código?
- ¿Tiene comentarios útiles?
- ¿Es complicado ampliar las funcionalidades?
- ¿Se puede probar fácilmente?
- Si queremos ejecutar este código en un servidor web, ¿Nos valdría?
- ¿Qué otros aspectos mejorarías?

Ejercicio preliminar

1. Crea un proyecto **público** en el servidor de gitlab de la universidad llamado “PracticaOOP”
2. Clona ese proyecto a tu ordenador
3. Crea dentro del proyecto un fichero con el código anterior. Llama a ese fichero “empleados.py”
4. Incorpora ese fichero al repositorio. Asegúrate que subes los cambios a gitlab.
5. Comparte la dirección del proyecto con tus compañeros
6. Trabajando en equipo crea tantos “Issues” como fallos o mejoras creáis razonables

Ejercicio 1

1. Sobre el proyecto “PracticaOOP” modifica el código anterior y crea una clase llamada Empleado que tenga:
 - a. Un nombre
 - b. Una nómina
2. Por el momento, no cambies el resto del código. Solo adáptalo para que use esa clase.
3. Sube tu código al repositorio de Gitlab

Paso 1.

```
def CalculoImpuestos (nombre, nom):
    impuestos = nom*0.30
    print ("El empleado {name} debe pagar {tax:.2f}".format(name=nombre,
    tax=impuestos))
    return impuestos

def displayCost(total):
    print("Los impuestos a pagar en total son {:.2f} euros".format(total))

class Empleado:
    """Un ejemplo de clase para Empleados"""
    def __init__(self, n, s):
        self.nombre = n
        self.nomina = s

empleadoPepe = Empleado("Pepe", 20000)
empleadaAna = Empleado("Ana", 30000)
displayCost(CalculoImpuestos(empleadoPepe.nombre, empleadoPepe.nomina) +
CalculoImpuestos(empleadaAna.nombre, empleadaAna.nomina))
```

Ejercicio 2

1. Modifica el código anterior. Modifica la función `CalculoImpuestos` para que sea un método de la clase `Empleado`.
 - a. Observa que existe una la diferencia entre los conceptos función y método
2. Por el momento, no cambies el resto del código. Solo adáptalo para que utilice ese método.
3. Sube tu código al repositorio de Gitlab

Paso 2.

```
def displayCost(total):  
    print("Los impuestos a pagar en total son {:.2f} euros".format(total))  
  
class Empleado:  
    """Un ejemplo de clase para Empleados"""  
    def __init__(self, n, s):  
        self.nombre = n  
        self.nomina = s  
  
    def CalculoImpuestos (self):  
        impuestos = self.nomina*0.30  
        print ("El empleado {name} debe pagar {tax:.2f}".format(name=self.nombre,  
            tax=impuestos))  
        return impuestos  
  
empleadoPepe = Empleado("Pepe", 20000)  
empleadaAna = Empleado("Ana", 30000)  
displayCost(empleadoPepe.CalculoImpuestos() +  empleadaAna.CalculoImpuestos())
```

Ejercicio 3

1. Modifica el código anterior. Convierte la segunda función en un método dentro de la clase Empleado
2. Por el momento, no cambies el resto del código. Solo adáptalo para que utilice ese método.
3. Vuelve a subir tu código al repositorio de Gitlab
4. Comprueba que puedes acceder a todos los commits anteriores

Paso 3.

```
class Empleado:
    """Un ejemplo de clase para Empleados"""
    def __init__(self, n, s):
        self.nombre = n
        self.nomina = s

    def calculo_impuestos(self):
        impuestos = self.nomina*0.30
        return impuestos

    def imprime(self):
        tmp = self.calculo_impuestos()
        print ("El empleado {name} debe pagar {tax:.2f}".format(name=self.nombre,
            tax=tmp))

empleadoPepe = Empleado("Pepe", 20000)
empleadaAna = Empleado("Ana", 30000)

total = empleadoPepe.calculo_impuestos() + empleadaAna.calculo_impuestos()

empleadoPepe.imprime()
empleadaAna.imprime()
print("Los impuestos a pagar en total son {:.2f} euros".format(total))
```

Ejercicio 4

1. Utiliza el método `__str__` para evitar que la clase haga entrada/salida.
2. Adapta el resto del código para que utilice ese método.
3. Sube tu código al repositorio de Gitlab y comprueba que existe un nuevo commit

Paso 4.

```
class Empleado:
    """Un ejemplo de clase para Empleados"""
    def __init__(self, n, s):
        self.nombre = n
        self.nomina = s

    def calculo_impuestos (self):
        return self.nomina*0.30

    def __str__(self):
        return "El empleado {name} debe pagar {tax:.2f}".format(name=self.nombre,
                                                                tax=self.calculo_impuestos())

empleadoPepe = Empleado("Pepe", 20000)
empleadaAna = Empleado("Ana", 30000)

total = empleadoPepe.calculo_impuestos() +  empleadaAna.calculo_impuestos()

print(empleadoPepe)
print(empleadaAna)
print("Los impuestos a pagar en total son {:.2f} euros".format(total))
```

Pruebas unitarias

Las pruebas unitarias son fragmentos de código que prueban el código de una aplicación.

Normalmente prueban elementos básicos tales como funciones aisladas, clases, métodos, etc.

Siempre debemos buscar que nuestra aplicación supere todas sus pruebas unitarias.

Os recomendamos algunas lecturas ya existentes en internet. Por ejemplo:
<https://realpython.com/python-testing/>

Ejemplo de pruebas unitarias

mul.py

```
def multiply(a, b):  
    return a*b
```

```
def add(a, b):  
    return a+b
```

multests.py

```
import unittest  
from mul import multiply  
  
class MultiplyTestCase(unittest.TestCase):  
    def test_multiplication_with_correct_values(self):  
        self.assertEqual(multiply(5, 5), 25)  
  
if __name__ == '__main__':  
    unittest.main()
```

Para Probar

```
import unittest

class PruebaUnitaria(unittest.TestCase):
    def test(self):
        pass

if __name__ == "__main__":
    unittest.main()
```


Para Probar (II)

```
import unittest

def numero(a): return a + 20

def alumno(nombre): return nombre

class PruebasUnitarias(unittest.TestCase):
    def test_numero(self):
        self.assertEqual(numero(5), 20)

    def test_alumno(self):
        self.assertIsNotNone(alumno(None))

if __name__ == '__main__':
    unittest.main()
```

Para Probar (III)

```
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

Ejemplo extraído de <https://docs.python.org/3/library/unittest.html>

Criterios de prueba

Método	Comprueba que	Nuevo en
<code>assertEqual(a, b)</code>	<code>a == b</code>	
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	
<code>assertTrue(x)</code>	<code>bool(x) is True</code>	
<code>assertFalse(x)</code>	<code>bool(x) is False</code>	
<code>assertIs(a, b)</code>	<code>a is b</code>	3.1
<code>assertIsNot(a, b)</code>	<code>a is not b</code>	3.1
<code>assertIsNone(x)</code>	<code>x is None</code>	3.1
<code>assertIsNotNone(x)</code>	<code>x is not None</code>	3.1
<code>assertIn(a, b)</code>	<code>a in b</code>	3.1
<code>assertNotIn(a, b)</code>	<code>a not in b</code>	3.1
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>	3.2
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>	3.2

Ejercicio 5

1. Reduce al mínimo el código de prueba que hay “a pelo” introduciendo pruebas unitarias
2. Sube tu código al repositorio de Gitlab y comprueba que existe un nuevo commit

Paso 5 (I)

```
#!/usr/bin/env python3
```

```
import unittest
```

```
class Empleado:
```

""Un ejemplo de clase para Empleados""

```
def init (self, n, s):
```

```
self.nombre = n
```

```
self.nomina = s
```

```
def calcula impuestos (self):
```

```
return self.nomina*0.30
```

```
def str (self):
```

```
return "El empleado {name} debe pagar {tax:.2f}".format(name=self.nombre,
    tax=self.calcula impuestos())
```

Paso 5 (II)

```
class TestEmpleado(unittest.TestCase):

    def test_construir(self):
        e1 = Empleado("nombre", 5000)
        self.assertEqual(e1.nomina, 5000)

    def test_impuestos(self):
        e1 = Empleado("nombre", 5000)
        self.assertEqual(e1.calcula_impuestos(), 1500)

    def test_str(self):
        e1 = Empleado("pepe", 50000)
        self.assertEqual("El empleado pepe debe pagar 15000.00", e1.__str__())

if __name__ == "__main__":
    unittest.main()
```

Nueva definición de los requisitos

Las personas tienen un nombre y un NIF (optativo).

Los empleados son personas que pagan impuestos según una tasa del 30%. Los impuestos se calculan sobre la nómina del empleado. Cada empleado tiene una nómina diferente.

Los jefes son empleados que tienen un bonus (paga extra) adicional. Pagan la misma tasa de impuestos sobre la suma de su nómina y el bonus.

Se pide hacer un programa que calcule la nómina de dos personas (Pepe y Ana) que ganan 20.000€ y 30.000€ respectivamente. Ana es una jefa que tiene un bonus de 2.000€

Ejercicio 6

1. Modifica el código anterior para adaptarlo a los nuevos requisitos
 - a. Primero modela los conceptos de Empleado y Jefe
 - b. En un segundo paso, modela el concepto de Persona
2. Sube tu código al repositorio de Gitlab y comprueba que existe un nuevo commit

Paso 6 (I)

```
class Empleado:
    """Un ejemplo de clase para Empleados"""
    def __init__(self, n, s):
        self.nombre = n
        self.nomina = s

    def calcula_impuestos (self):
        return self.nomina*0.30

    def __str__(self):
        return "El empleado {name} debe pagar {tax:.2f}".format(name=self.nombre,
            tax=self.calcula_impuestos())
```

Paso 6 (II)

```
class Jefe(Empleado):

    def __init__(self, n, s, extra=0):
        super().__init__(n, s)
        self.bonus = extra

    def calcula_impuestos1 (self):
        return (self.nomina + self.bonus)*0.30

    def calcula_impuestos (self):
        return super().calcula_impuestos() + self.bonus*0.30

    def __str__(self):
        return "El jefe {name} debe pagar {tax:.2f}".format(name=self.nombre,
            tax=self.calcula_impuestos())

e1 = Empleado("Pepe", 20000)
j1 = Jefe("Ana", 30000, 2000)

print(e1)
print(j1)
```

Ejercicio 7

1. Modifica el código anterior para que respete todos los requisitos (introduce el concepto de Persona)
2. Haz pruebas unitarias. Intenta añadir todas las pruebas unitarias que puedas
3. Sube tu código al repositorio de Gitlab
4. Avisa al profesor para que podamos corregirlo en clase.

(En este caso, no damos la solución)

Ejercicio 8

1. Separa tu código en varios ficheros:
 - a. Un fichero por cada clase. El nombre del fichero debería corresponder con el nombre de la clase. Por ejemplo: Persona.py tendría la clase Persona
 - b. Un fichero con todas las pruebas
2. Sube el código a gitlab
3. Avisa al profesor para que podamos corregirlo en clase.

(De nuevo, no damos la solución)

Pruebas automáticas

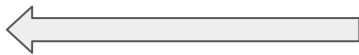
Gitlab puede hacer las pruebas por tí.

Crea un fichero llamado “.gitlab-ci.yml” (tiene un punto al principio)

En ese fichero introduce el siguiente contenido:

```
stages:  
  - test
```

```
test-unittest:  
  stage: test  
  script:  
    - python3 -m elficherodetest
```



Cambia elficherodetest por el nombre del fichero que tenga los tests

Pruebas automáticas

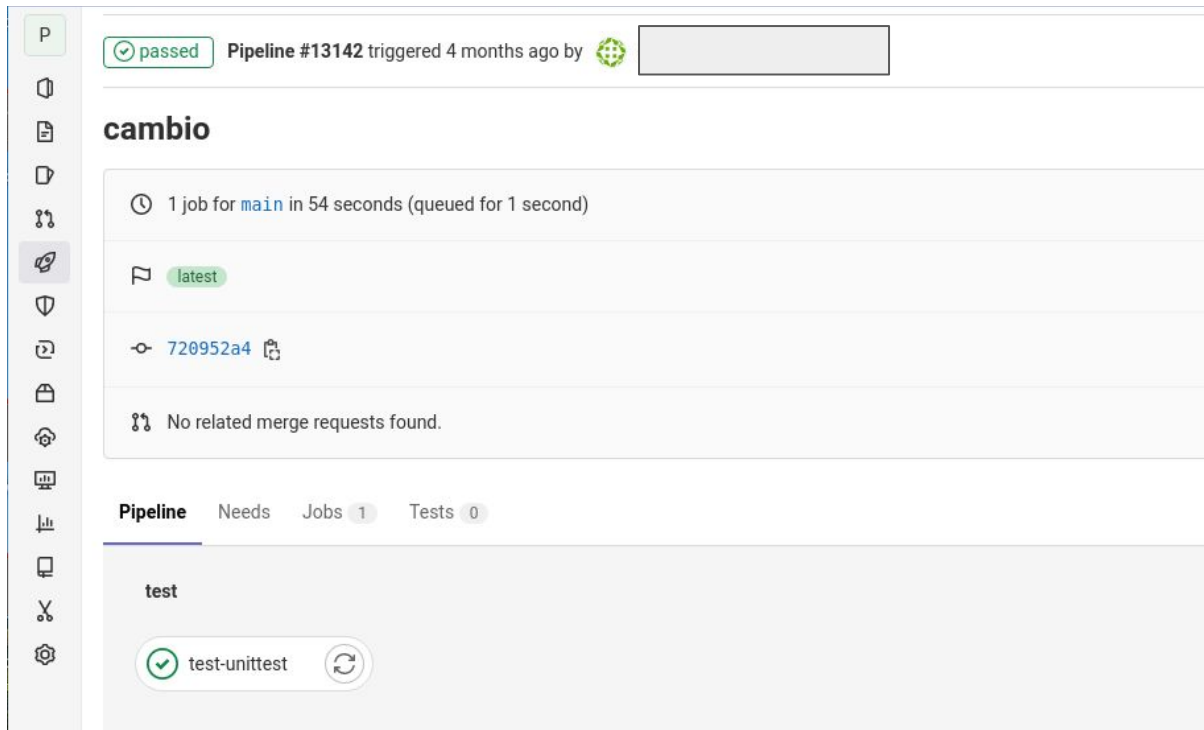
Ejemplo

 `.gitlab-ci.yml`  94 bytes

Edit in pipeline editor ▼ Lock Replace Delete   

```
1 ---
2
3 stages:
4   - test
5
6 test-unittest:
7   stage: test
8   script:
9     - python3 -m test_servidor
10
11
```

Gitlab lanzará las pruebas



The screenshot displays the GitLab CI/CD interface. On the left is a sidebar with navigation icons, including a 'P' icon at the top. The main content area shows a summary of a pipeline:

- A green box with a checkmark and the word "passed" is followed by the text "Pipeline #13142 triggered 4 months ago by" and a user icon.
- The word "cambio" is displayed in bold.
- A clock icon is followed by the text "1 job for main in 54 seconds (queued for 1 second)".
- A flag icon is followed by a green box containing the word "latest".
- A key icon is followed by the commit ID "720952a4" and a document icon.
- A merge request icon is followed by the text "No related merge requests found."

Below this summary is a tabbed interface with the following tabs: "Pipeline", "Needs", "Jobs 1", and "Tests 0". The "Pipeline" tab is currently selected. It shows a job named "test" with a sub-job "test-unittest" that has a green checkmark icon and a circular refresh icon.

Ejercicio 9

1. Activa los mecanismos de integración continua en gitlab
2. Comprueba que Gitlab hace las pruebas unitarias
3. Avisa al profesor para que podamos corregirlo en clase
4. Si todo es correcto, revisa los Issues que abrimos al principio y ciérralos si crees que hemos cumplido los objeti

(Siguiendo nuestra línea, no damos la solución)